DOCUMENT 01 / SMART UNIVERSITY DIGITAL CAMPUS

Detailed Solution Synopsis

UniSphere Smart University Digital Campus | An integrated student journey with accountable administration, connected services, and assisted decision-making.

Solution at a glance A functional Flutter Windows prototype and FastAPI backend connect student, administrator, and parent journeys through shared institutional records. The current stage demonstrates core workflows with local demo data; institution-wide adoption requires source-data approval, security review, and a controlled pilot.

The campus problem

University services are often split among offices, spreadsheets, separate portals, and manual follow-up. A student may need one office for enrolment, another for attendance or exams, and more visits for accommodation, transport, fees, or a certificate. Staff face repeated data entry and incomplete views of a student's situation. Parents need a permitted, reliable view rather than informal updates. Fragmentation slows service and weakens accountability.

The proposed solution

UniSphere gives each user an authenticated role and connects academic administration, service operations, certificates, support, and student-success signals to one student identity. The university remains the owner of records and decisions. Admin actions create or update the data; students and linked parents see the relevant views after login or refresh. The assistant helps explain records and navigate services, while officers approve, issue, reconcile, and intervene.

User	Main need	UniSphere response
Student	Find accurate personal information and finish campus tasks.	Personal dashboard, academics, timetable, attendance, exams, services, certificates, support and AI assistance.
Parent	See an authorized view of a linked student's progress.	Staff-created relationship link and student-specific academic and financial overview.
Administrator	Maintain records and resolve requests with clear ownership.	Master data, Student 360, academic operations, service allocation, certificate review, risk and intervention queues.

Design principle

The platform follows an operational chain: approved university master data -> student registration -> academic assignment -> attendance, examinations and services -> authorized student/parent views -> staff-led support. A new account does not create fictitious courses or allocations. Missing records should remain clear empty states.

End-to-end campus workflow

Stage	Responsible action	Visible result
1. University setup	Admin imports or creates departments, programmes, specializations, sessions, curricula and courses.	A governed catalogue for registration and academic planning. Demo curricula require institutional sign-off.
2. Student identity	Admin registers a student and the backend associates the account with a unique student identifier.	Student login resolves this student's data, not a hard-coded sample profile.
3. Academic plan	Admin assigns programme, specialization, session and semester; optional provisioning creates course enrolments, timetable slots, an invoice and demo exam plan.	Student academics, timetable and related views show assigned records.
4. Academic activity	Authorized staff manage attendance sessions, examinations, registrations and results.	Student, parent and admin views draw from the recorded activity.
5. Campus services	Hostel/transport officers configure inventory and allocate a room or route to a student.	The student sees their building and room or route and pass, and can report an issue.
6. Certificate	Student requests a certificate; authorized staff review, approve/reject, issue or revoke it.	A numbered PDF and verification status become available after issuance.
7. Parent relationship	Admin creates a parent account and formally links one or more students.	The parent sees only linked students through the parent portal.
8. Student support	Students ask for help; staff review warnings and document interventions.	Request status and support follow-up are traceable in the relevant workflow.

Connected-record example The captured Windows prototype shows Raj in an assigned academic plan, the admin hostel occupancy view, and Raj's student hostel view. The same Shaligram room A1 allocation appears in both roles. This demonstrates a connected local record; the evaluator can repeat the route using Document 08.

Capabilities by portal

Portal / area	Current functional scope
Admin governance	University and academic master data; student registration; Student 360; academic planning; dashboards and report views.
Academic operations	Semester/course/offering and enrolment administration; timetable; attendance sessions and bulk marking; examination scheduling, registration and results.
Fee operations	Fee structures, generated invoices, adjustments and manually recorded payments. Live gateway settlement is outside the verified prototype.
Service operations	Hostel buildings, rooms, capacity, allocations and requests; transport routes, stops, vehicles, passes and issues.
Certificate desk	Student service-certificate requests, staff review, issue, PDF/QR verification and revocation. Public verification needs a deployed domain.
Student portal	Identity-linked dashboard, courses, attendance, timetable, exams/results, fees, hostel, transport, certificates, helpdesk, AI and student-success views.
Parent portal	Staff-controlled parent registration/link; linked-student selector and live attendance, fee, exam and attention overview.
Support and intelligence	Helpdesk and notification workflows; rule-based risk scoring, early warnings, interventions and assistant answers.

What makes the solution distinctive

- **One student identity across roles:** the admin creates and maintains a record that appears in the correct student's portal and, when linked, in a parent view.
- **Transactions before dashboards:** academic plans, allocations, certificate approvals and intervention actions have staff-owned workflows.
- **Assistance anchored in records:** personal answers come from authenticated student context; general questions can use curated knowledge or a model provider.
- **Explainable support signal:** the risk score states its contributing factors and leads to human review rather than an automatic decision.

Current administrative boundaries

Grievance, dedicated admin Voice Assistant, admin Academic Passport, secure sharing, permission configuration, audit-log UI, realtime administration and settings are marked unavailable in the current admin navigation. They are not presented here as delivered workflows.

AI and student-success approach

The assistant routes personal questions to authenticated records such as fees, attendance, timetable, certificates and service allocations. Campus-policy questions can use approved guidance; simple general questions can use curated responses; unanswered open questions can reach Gemini through a backend-held key. The client offers English/Hindi choices and voice input/output. It does not train a custom language model or demonstrate all Indian languages.

Conversation history can contain prior personal answers and may be forwarded in an external model prompt. The direct student-context payload is excluded from external requests, but history redaction, consent, retention and provider review are required before using institutional data in production.

Student-success element	Current behavior	Interpretation
Risk score	Weighted attendance 40%, academic 30%, fee 15%, engagement 15%; low/medium/high bands.	Explainable rule-based signal, not a validated predictive model.
Early warning	Signals are shown for staff review with severity and student context.	A warning suggests outreach; it is not proof of future dropout.
Intervention	Authorized staff can assign and track a support response.	Human judgement and documented follow-up remain central.

Technical architecture

Layer	Technology and role
Client	Flutter/Dart portals; Windows desktop is the demonstrated target. Other targets need device acceptance testing.
API and access	FastAPI REST, Pydantic validation, JWT role checks and per-request authenticated user lookup.
Data	SQLAlchemy relational models; SQLite for local demo; PostgreSQL in a production Compose example.
Documents and AI	ReportLab certificate PDFs/QR verification; Gemini adapter called from the backend for eligible general questions.

This is a modular application architecture with one backend as the system boundary. Production operation requires HTTPS, secure client token storage, backup/restore, access testing, logging and workload tests; these are rollout conditions, not implied by the demo.

Evidence, impact and adoption path

The proof of concept is a working local Windows client connected to backend workflows and demonstration records. The strongest observable path is admin academic planning -> admin hostel allocation -> Raj's student view -> certificate status -> student assistant. The separate prototype document contains screen captures and a repeatable seven-minute demo. The screenshots show existing state, not measured university-wide outcomes.

Evidence type	What it establishes	What remains to establish
Windows UI captures	Multiple portals and account-specific records can be inspected.	Independent device/browser acceptance and accessibility.
Backend code and routes	Registration, service, certificate, AI and risk workflows exist.	Institutional data quality, security review, live operational support.
Demo catalogue	Programme/curriculum imports can populate the workflow.	Official syllabus and credit approval by academic owners.
Pilot measures	A method can assess completion, turnaround and correctness.	Baseline and observed results; no percentage improvement is yet claimed.

Expected value

A successful campus pilot could reduce repeated office visits and staff re-entry, improve transparency of request status, help students act on current academic information, give parents permission-controlled visibility, and support earlier staff outreach. These are expected benefits. The project has not measured time savings, satisfaction, graduation outcomes or model accuracy.

Practical next step

Pilot one approved programme or department with a limited student cohort. Name the registrar, academic, finance, certificate and service owners; reconcile source records; run authorization and restore tests; then compare task completion, service turnaround, record agreement, support response and user experience against a baseline. Expand only when the measured results and controls meet agreed gates.

Evaluator conclusion UniSphere is more than a static interface: a connected operational prototype already ties admin-maintained records to student and parent experiences. Its strongest claim today is a credible, testable workflow foundation for a controlled digital-campus pilot.

Evidence base

Repository: **EXECUTIVE_SUMMARY.md**, Flutter **lib/**, FastAPI **backend/app/**, deployment configuration and the captured Windows prototype. Companion PDFs: Documents 05-08 for impact, implementation, technology and screenshot-backed proof. All scope statements reflect the reviewed local prototype as of 1 October 2026.